

SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1-INTERACTIVE TREES SIMULATOR

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO3-Precision (BL3, BL5)	Assessment:	Assessment Tool 1-Interactive Tree Simulator
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees.		
Student Name:	K. Nilavan	Reg. No.:	192511028
Section/Batch:	2 ND Year CSE	Date:	27/07/2026

Code of Conduct

I, K. Nilavan (192511028) (Name/RegNo) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Nilavan

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answers using simulator, in evaluation the answers should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section/Topic	Focus	QNos	Marks
Binary Search Tree	Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals	Traversal types, Inorder, Postorder and Preorder	2	20
AVL Tree	Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree	Properties, structures and Operations	4	20
Red Black Tree	Properties, Example and Operations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Interactive Tree Simulator

- Construct a Binary Search Tree (BST) by inserting the following keys: 40, 20, 10, 30, 60, 50, 70. Perform:
 - Inorder Traversal
 - Search for 70
 - Delete 50
 - Display the final BST.
- Complete Tree Traversals by inserting 55, 35, 75, 25, 45, 65, 85. Find all the traversals.
 - Inorder
 - Preorder
 - Postorder
- Insert 40, 20, 60, 10, 30, 50, 70, 5 into an AVL Tree. Determine the balance factor of every node before and after balancing.
- Construct a B-Tree of order 3 by inserting 10, 20, 30, 40, 50.
- Construct a Red-Black Tree using 50, 40, 60, 30, 45, 55, 70, 20. Delete 30 and analyze how Red-Black properties are restored.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations
Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, nodes split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUESTION-1:

SEE ALGORITHMS

Articles About Sign in

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black Tree

```
else:  
    insert(node.right, key)
```

Enter a number: 59 ▶ INSERT DELETE CLEAR ↶ ↷ ↺ ↻ 📄 🔗

```
graph TD  
    40((40)) --> 20((20))  
    40 --> 60((60))  
    20 --> 10((10))  
    20 --> 30((30))  
    60 --> 50((50))  
    60 --> 70((70))
```

Controls

Input
Input is level-by-level order.

40,20,10,30,60,50,70

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre In Post

Generate Default

Visualize

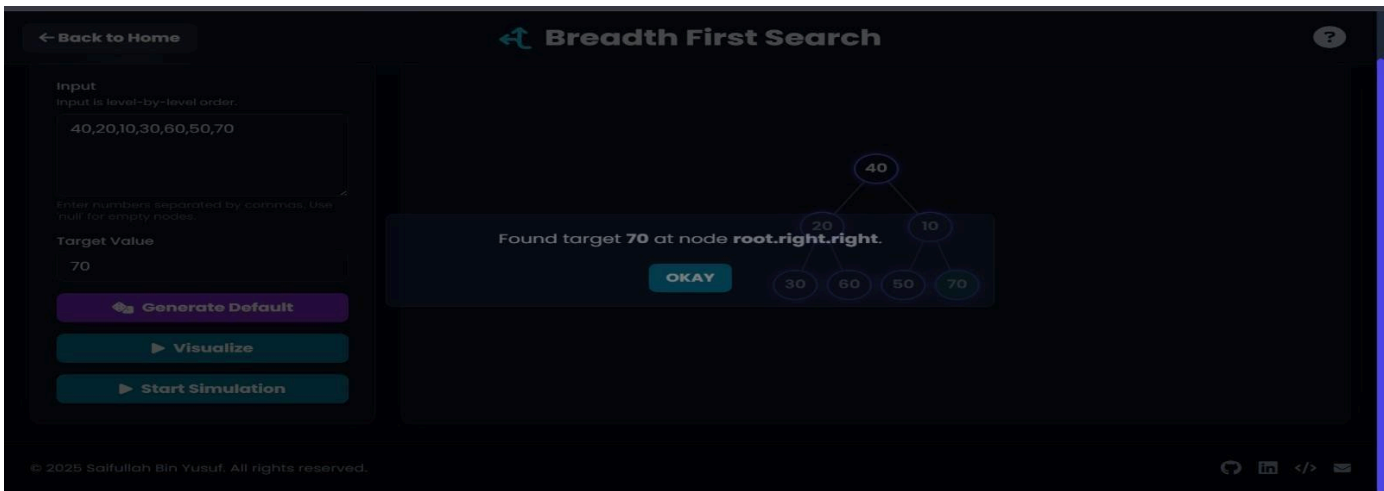
Start Simulation

30 20 60 40 50 10 70

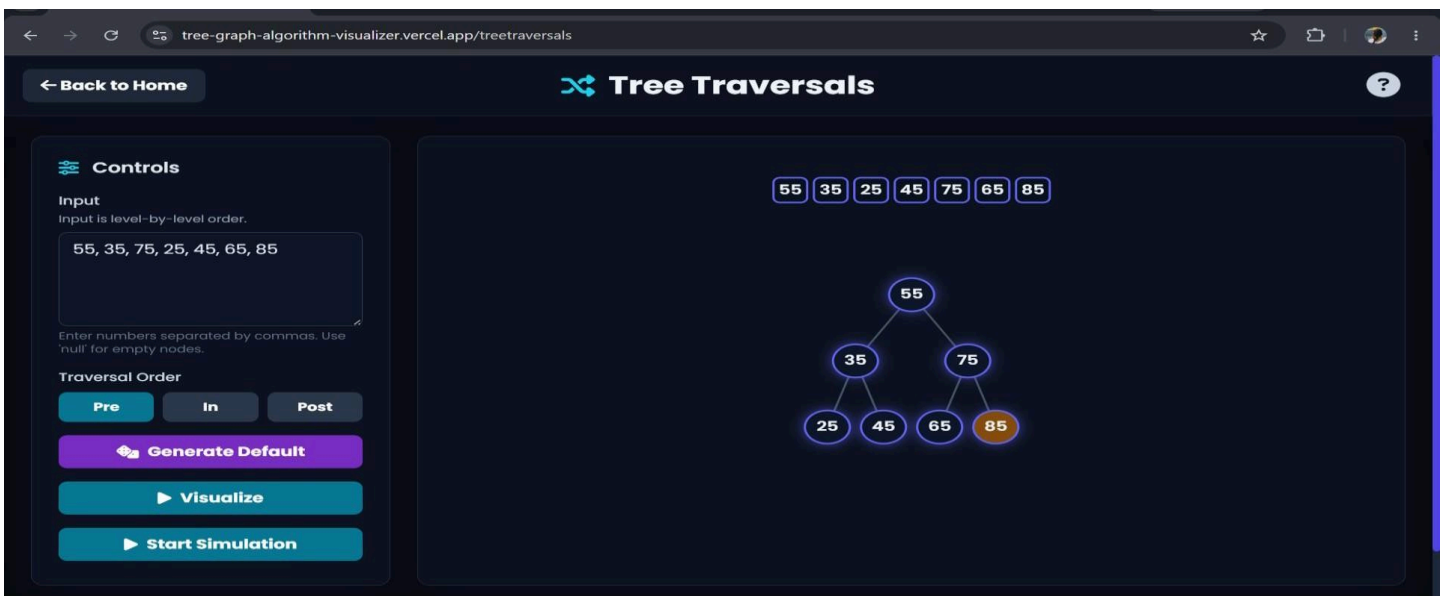
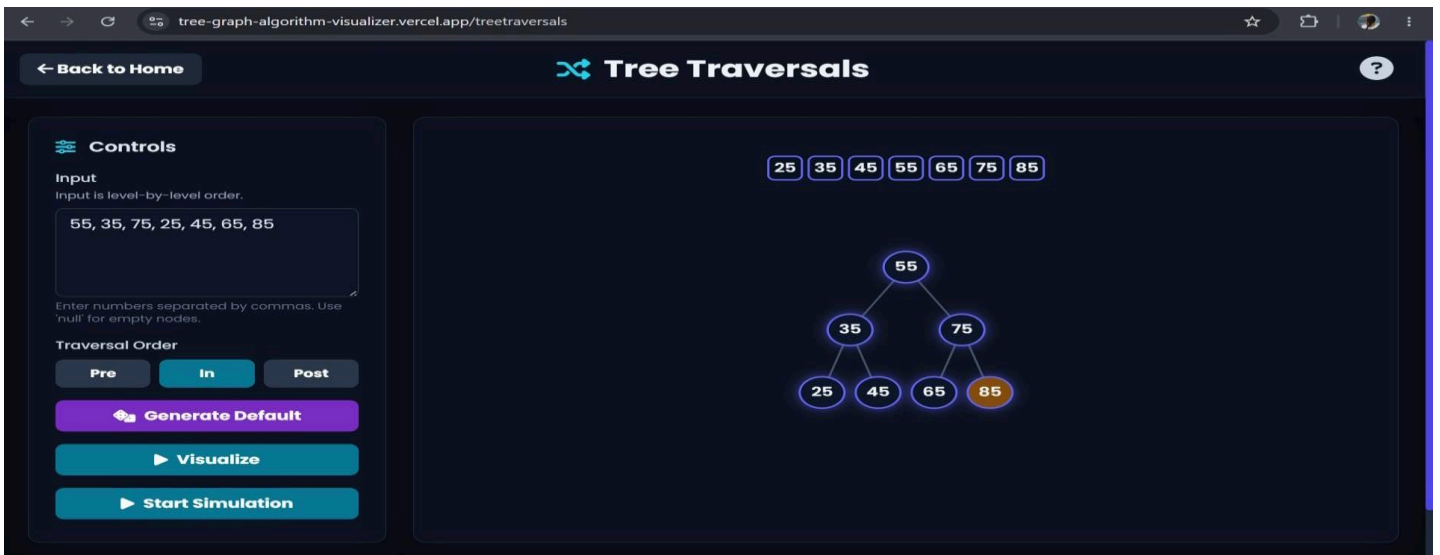
```
graph TD  
    40((40)) --> 20((20))  
    40 --> 10((10))  
    20 --> 30((30))  
    20 --> 60((60))  
    10 --> 50((50))  
    10 --> 70((70))
```

Enter a number: 46 ▶ INSERT DELETE CLEAR ↶ ↷ ↺ ↻ 📄 🔗

```
graph TD  
    40((40)) --> 20((20))  
    40 --> 60((60))  
    20 --> 10((10))  
    20 --> 30((30))  
    60 --> 50((50))  
    60 --> 70((70))
```



QUESTION-2



[← Back to Home](#)

Tree Traversals
?

Controls

Input

Input is level-by-level order.

55, 35, 75, 25, 45, 65, 85

Enter numbers separated by commas. Use null for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

25 45 35 65 85 75 55

QUESTION-3

SEE ALGORITHMS

[Articles](#)
[Ab](#)

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

rebalance(node.parent)

Enter a number: 21

▶

INSERT

DELETE

CLEAR

↶

↷

💾

🔗

QUESTION-4

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

or keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

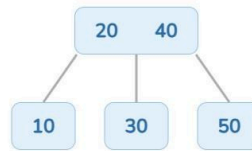
Enter a number: 53



INSERT

SEARCH

CLEAR



QUESTION-5

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

leaves (null nodes) are black, **red nodes cannot have red children**, and every path from a node to its descendant leaves contains the same number of black nodes.

recoloring to restore properties.

4. Repeat until all rules are satisfied from the modification to the root.

Enter a number: 21



INSERT

CLEAR

